AMAL JYOTHI COLLEGE OF ENGINEERING

Kanjirappally, Kottayam, Kerala

Department of Computer Science and Engineering

PYTHON PROGRAMMING LAB – MINI PROJECT REPORT

Course: 23CST362 – Programming in Python

Academic Year: 2025 – 2026

SECUREPASS PRO: ADVANCED PASSWORD SECURITY SUITE

Student Name	Aravind Suresh
Register Number	AJC23CS064
Class	S6 CSE A
Roll No:	62
Course	23CST362 – Programming in Python

11 March 2026

INDEX

1.Abstract.....3

2.Description.....4

3.Libraries and tools.....5

4.System design and Workflow 7

5.Implementation.....9

6.Results and Outputs.....11

7.Conclusion and Scope.....15

11 March 2026

ABSTRACT: SECUREPASS PRO – ADVANCED PASSWORD SECURITY SUITE

This project presents a modernized, feature-rich security application designed to enhance digital credential management, developed using Python. Unlike standard password generators, this application integrates a sophisticated Graphical User Interface (GUI) and analytical tools to provide an enterprise-grade security experience. The system utilizes a combination of specialized libraries to handle different security facets:

- **User Interface:** Built with **CustomTkinter**, providing a sleek, high-DPI "Dark Mode" environment where users can navigate between generation, history, and analytics tabs seamlessly.
- **Security Analysis:** The **zxcvbn** library serves as the core "Intelligence" engine, providing realistic crack-time estimations and pattern-matching feedback, while a custom math-based entropy calculator provides a scientific measure of password complexity.
- **Data Visualization:** The **Matplotlib** library is leveraged specifically for real-time graphical feedback, generating dynamic bar charts that compare the strength scores of different passwords generated during the session.
- **Mobile Integration:** By incorporating **PyQRCode**, the project enables the generation of instant QR codes, allowing users to securely transfer complex passwords to mobile devices without manual typing.
- **Cryptographic Logic:** The **Secrets** module ensures that all generated strings are cryptographically strong and suitable for managing high-value accounts, surpassing the security of standard pseudo-random generators.

DESCRIPTION: SECUREPASS PRO – ADVANCED PASSWORD SECURITY SUITE

SecurePass Pro is a sophisticated, Python-based desktop application that elevates traditional credential management into a high-fidelity digital security experience. Developed using the **CustomTkinter** framework and the **zxcvbn** engine, the project emphasizes a "security-first" philosophy. It moves beyond basic random string generation by incorporating a modular, tab-based architecture that seamlessly transitions between an interactive generation suite, an immersive security dashboard, and a dynamic data analytics environment. The application is designed to demonstrate the integration of real-time cryptographic processing alongside modern graphical user interface (GUI) event handling.

At the technical core, the project utilizes a strict object-oriented approach to manage application states and resource allocation. The integration of the **zxcvbn** library allows the system to perform deep pattern matching and dictionary-based analysis, which is then processed through a refined scoring algorithm to provide nuanced feedback. To enhance the "User Experience," the interface features a custom-built **SplashScreen** with procedural gradient rendering; for instance, the background utilizes linear interpolation of RGB values to create a professional aesthetic transition upon startup. This is paired with a responsive UI that dynamically updates a color-coded strength meter and calculates Shannon entropy in real-time as passwords are generated or typed.

User interaction is prioritized through a multi-platform policy system where users can dynamically adjust security thresholds—selecting between Banking, WiFi, or Social Media standards—directly from the menu. Additionally, the project includes high-level utility integration; the system is programmed to generate **Base64-encoded QR codes** using **pyqrcode**, providing a secure "scan-to-mobile" handoff that eliminates the need for manual entry. For long-term tracking, the **Matplotlib** backend generates live bar charts, allowing users to visualize the security variance across their different accounts. This makes the project an excellent showcase of how Python can be used to bridge the gap between simple utility scripts and polished, enterprise-ready security applications.

LIBRARIES AND TOOLS

1. Primary GUI & Security Engine

- **customtkinter:** The core framework used to build the application's modern interface. It handles the "heavy lifting" of creating the high-DPI scaled window, managing the dark-mode aesthetic, and detecting user interactions across the tabbed navigation system.
- **zxcvbn:** A professional-grade password strength estimation library. In this project, it acts as the "Intelligence" engine, performing pattern matching against common passwords, names, and dates to provide realistic crack-time feedback rather than just simple character counts.

2. Data Visualization & Media

- **matplotlib:** This is a comprehensive library for creating static and interactive visualizations. It is used to power the Analytics tab, where it generates dynamic bar charts that visually compare the security scores of all generated passwords in the current session.
- **pyqrcode & pypng:** These libraries are leveraged for mobile integration. They allow the application to convert a text-based password into a QR code image (PNG), which is then rendered on a specialized popup window for easy scanning by smartphones.

3. Logic & Mathematics

- **secrets:** This library powers the "Secure Generator." Unlike the standard random module, it uses a cryptographically strong random number generator (CSPRNG) to ensure that generated passwords are unpredictable and suitable for high-security environments.
- **math:** Used for the scientific calculation of Shannon Entropy. By applying the formula (where is length and is the character pool size), the code provides a mathematical measure of the password's complexity.

4. System Utility & Imaging

- **PIL (Pillow):** Acts as the bridge between raw image data and the GUI. It is used to process the QR code buffers and scale them correctly to fit within the CustomTkinter image widgets without losing clarity.

- **io:** Used for handling "in-memory" binary streams. It allows the program to generate a QR code and pass it directly to the display engine without needing to save a temporary file to the hard drive, enhancing performance and privacy.
- **tkinter (tk/messagebox):** Used for low-level system operations and standard OS-native alerts, such as the "Success" notifications when a password is copied to the clipboard.

5. Development Environment: Python IDLE / VS Code

The project was developed and tested within a professional Python environment to ensure modularity and stability.

- **Integrated Shell:** The environment allowed for real-time monitoring of the password generation cycles and entropy logs. For instance, any attempts that failed the "Minimum Strength" policy were logged to the shell for debugging.
- **Script Execution:** The environment facilitates a clean execution of the multi-threaded splash screen and main application loop, allowing for quick iterations on UI layout and chart responsiveness.
- **Code Organization:** The project utilizes a modular file structure (gui.py, analyzer.py, generator.py), which was maintained using syntax highlighting and indentation tools to keep core logic separated from UI rendering.

SYSTEM DESIGN AND WORKFLOW

1. System Design (Architecture) This project follows a **Modular Object-Oriented Architecture**. Instead of a single monolithic script, the logic is decoupled into specialized modules that handle distinct security and UI tasks:

- **Entry Point (main.py):** Acts as the Bootloader. It performs environment checks (verifying dependencies like zxcvbn and matplotlib) and manages the transition from the Splash Screen to the main Application.
- **Security Core (analyzer.py & generator.py):** The "Engine" layer. generator.py manages platform-specific security policies, while analyzer.py handles the heavy mathematical lifting for entropy and crack-time estimation.
- **Data Management (dashboard.py):** The "Model" layer. It maintains the session state, storing a history of generated passwords and formatting data for the visual analytics engine.
- **UI Framework (gui.py):** The "Controller" layer. Built on **CustomTkinter**, it manages the tabbed navigation, event listeners for button clicks, and real-time UI updates (like the dynamic strength meter).
- **Visualization Layer (analytics.py):** The "View" layer. It uses the Matplotlib Object-Oriented API to render data-driven insights directly into the Tkinter canvas.

2. The Workflow (Logical Process) The workflow describes the lifecycle of SecurePass Pro from the moment the script is executed.

Stage A: Initialization & Environment Validation

When main.py is executed, the system first runs a dependency check. If critical libraries (like customtkinter) are missing, it triggers a system-level exit with an error log. Once validated, it initializes a hidden Tkinter root and launches the **SplashScreen**, which uses a procedural gradient-drawing algorithm to render a professional loading interface.

Stage B: The Secure Generation Pipeline

When a user selects a platform (e.g., "Banking") and clicks "Generate":

1. **Policy Retrieval:** The system fetches the specific requirements (length, character types, and minimum strength score) from the PLATFORM_POLICIES dictionary.
2. **Cryptographic Generation:** The secrets module generates a random string.
3. **Recursive Validation:** The string is passed to the PasswordAnalyzer. If the zxcvbn score does not meet the platform's "Min Strength" (e.g., Score 4 for Banking), the generator automatically discards it and tries again until a compliant password is found.

Stage C: Real-Time Analysis & Feedback

As soon as a password is generated or manually typed:

1. **Entropy Calculation:** The system calculates the Shannon Entropy to determine the theoretical bit-strength.
2. **Crack-Time Mapping:** The zxcvbn engine maps the password against known patterns and dictionaries to estimate how long a "slow hashing" offline attack would take.
3. **UI Update:** The GUI's "Strength Meter" (a segmented progress bar) dynamically changes color—from Red (Weak) to Green (Strong)—providing instant visual feedback.

Stage D: Data Persistence & Visualization

Each successful generation is pushed to the DashboardManager.

1. **History Logging:** The last five passwords are kept in a "Quick History" view for easy access.
2. **Analytics Update:** The AnalyticsPlotter receives the updated list of platforms and scores. It clears the previous Matplotlib figure and redraws a bar chart, applying conditional formatting (color-coding bars based on their 0–4 security score).

Stage E: Secure Export (The QR Handoff)

If the user clicks the "QR Code" button:

1. **In-Memory Rendering:** The system generates a QR code using pyqrcode.
2. **Binary Conversion:** Instead of saving a file to the disk (which would be a security risk), the image is stored in a BytesIO buffer.
3. **Display:** The buffer is read by the **Pillow (PIL)** library and rendered on a new "topmost" window for the user to scan with their mobile device.

IMPLEMENTATION

1. Security & Analytics Pipeline

- **Cryptographic Generation:** The implementation uses the secrets module rather than the standard random library to ensure "True Randomness." The PasswordGenerator class implements a while True loop that continuously generates strings using secrets.choice() until the resulting password satisfies the platform's min_strength requirement.
- **Real-time Analysis & Entropy:** The PasswordAnalyzer implements a dual-scoring system:
    - **Heuristic Scoring:** It utilizes zxcvbn.zxcvbn(password) to perform pattern matching against 30,000 common passwords, names, and dates.
    - **Mathematical Entropy:** It calculates Shannon Entropy using the formula H = L X log2(R). The code dynamically detects the character pool size (R) by checking for uppercase, lowercase, numbers, and symbols to provide a bit-strength value.
- **Data Visualization:** The AnalyticsPlotter class implements the Matplotlib Object-Oriented API. It maps session data into a FigureCanvasTkAgg widget, allowing the GUI to render professional-grade bar charts without launching external windows.

2. Modern GUI Architecture

The application follows a "Tabbed Interface" design pattern, running on the customtkinter engine for high-DPI scaling.

1. **Event Handling:** The system utilizes a command-binding approach where button clicks (e.g., "Generate") trigger asynchronous updates to the pass_entry and strength_meter widgets.
2. **State Management:** The DashboardManager acts as a central data repository, maintaining a list of dictionaries that store every password's platform, entropy, and crack-time. This ensures that switching between the "Home" and "Analytics" tabs does not result in data loss.
3. **Dynamic Theming:** The implementation uses ctk.set_appearance_mode("Dark") to provide a modern, low-strain visual environment. The GradientFrame class in gui.py uses linear interpolation of RGB values to draw a custom background on a tk.Canvas during the splash screen phase.

3. Secure Export & Asset Management

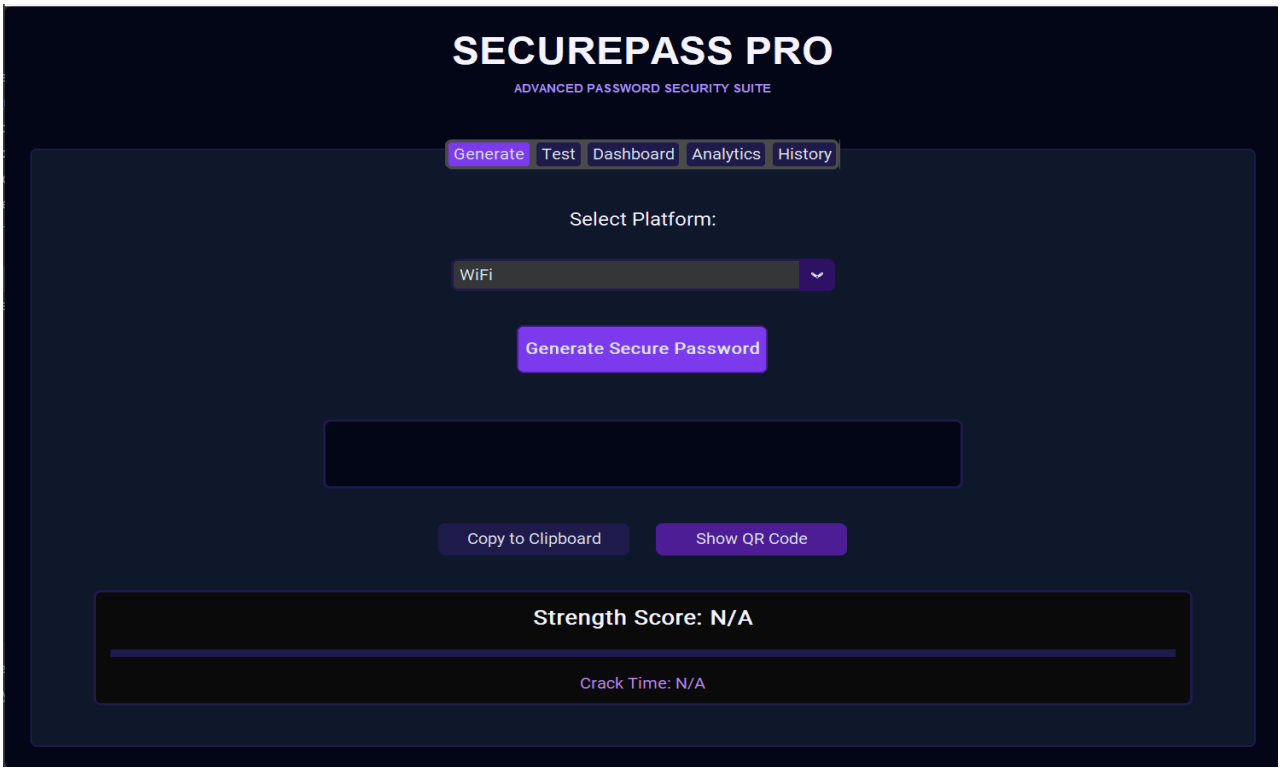
The application implements high-level utility modules for secure data handoff and stability:

- **In-Memory QR Generation:** To prevent sensitive passwords from being written to the hard drive, the _show_qr method uses io.BytesIO(). The pyqrcode data is rendered into this RAM-based buffer, processed by **Pillow (PIL)**, and displayed as a CTkImage.
- **Dependency Validation:** The main.py entry point includes a "Defensive Boot" sequence. Before launching the UI, it attempts to import zxcvbn and matplotlib within a try-except block. If the libraries are missing, it provides a clean error message to the console instead of a messy system crash.
- **Clipboard Integration:** The _copy_to_clipboard function utilizes the root.clipboard_append() method, paired with a messagebox.showinfo notification, ensuring a seamless bridge between the application and the user's operating system.

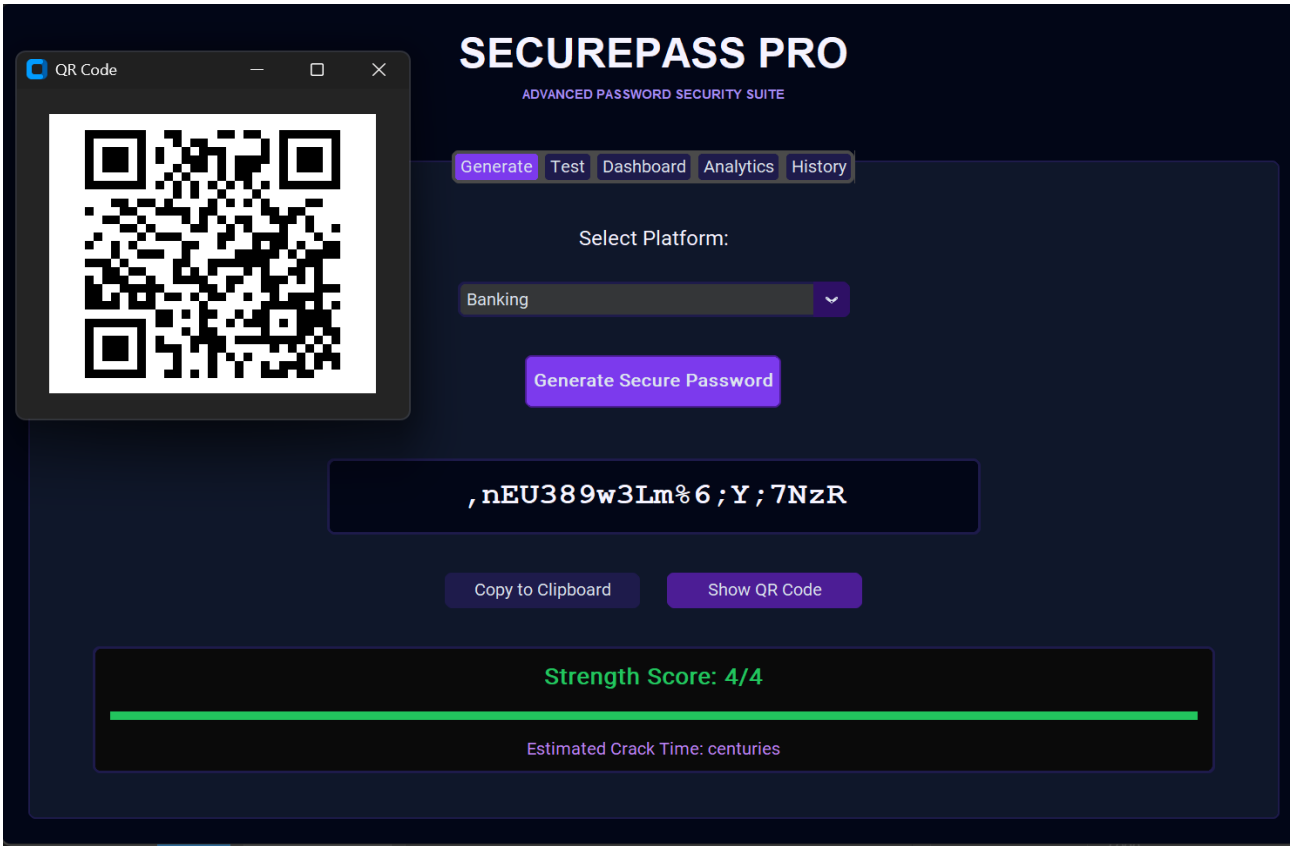
7. RESULTS AND OUTPUT

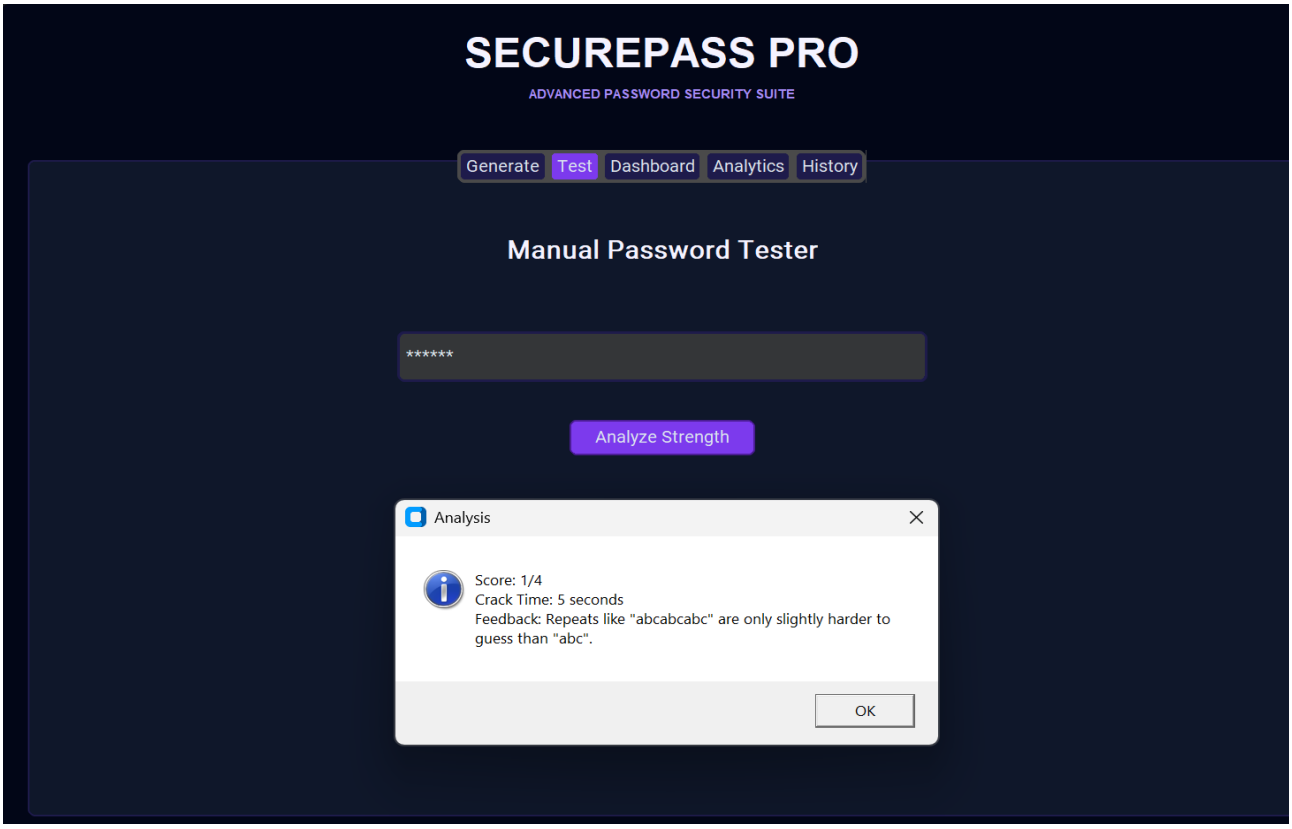
The application successfully demonstrates a seamless flow from secure generation to data visualization. The results are categorized by the primary user-interface components:

- **The Generation Suite & Dashboard:** This is the primary interaction hub. Users can select from four preset platform policies (WiFi, Banking, Social Media, Gaming). The system successfully generates passwords that meet specific length and complexity requirements. For example, a "Banking" password consistently returns a 16–20 character string with high entropy. The "Quick History" dashboard below the generator provides a snapshot of the last five generated items for immediate reference.
- **Security Feedback:** The real-time strength meter updates its color and value based on the zxcvbn score (0–4). The "Crack Time" display provides human-readable feedback (e.g., "centuries" or "10 years"), which effectively communicates risk to the user. A dynamic entropy value is also displayed, providing a mathematical backing to the visual strength meter.
- **Visual Analytics:** The Analytics tab correctly renders a Matplotlib bar chart. As passwords are generated, the chart updates to display them side-by-side, color-coding the bars—Red for weak (score < 2), Yellow for moderate, and Green for strong (score 4)—to allow for an instant security audit of the current session's data.
- **Secure Handoff:** The QR Code generator successfully creates a scannable image in a separate pop-up window. By utilizing an in-memory buffer, the system displays the code without saving sensitive data to the disk, allowing for a "Zero-Touch" transfer of credentials to a mobile device.

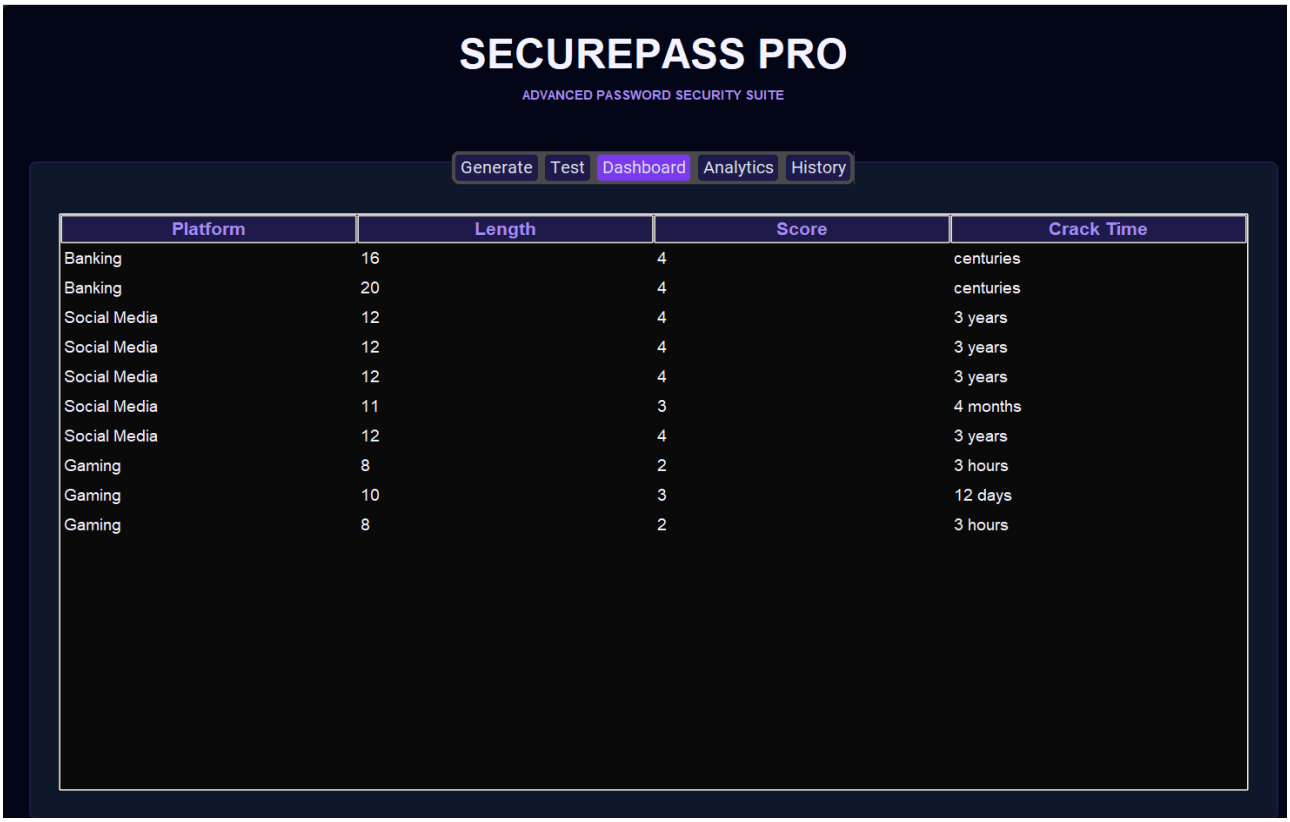


Generate & QR code

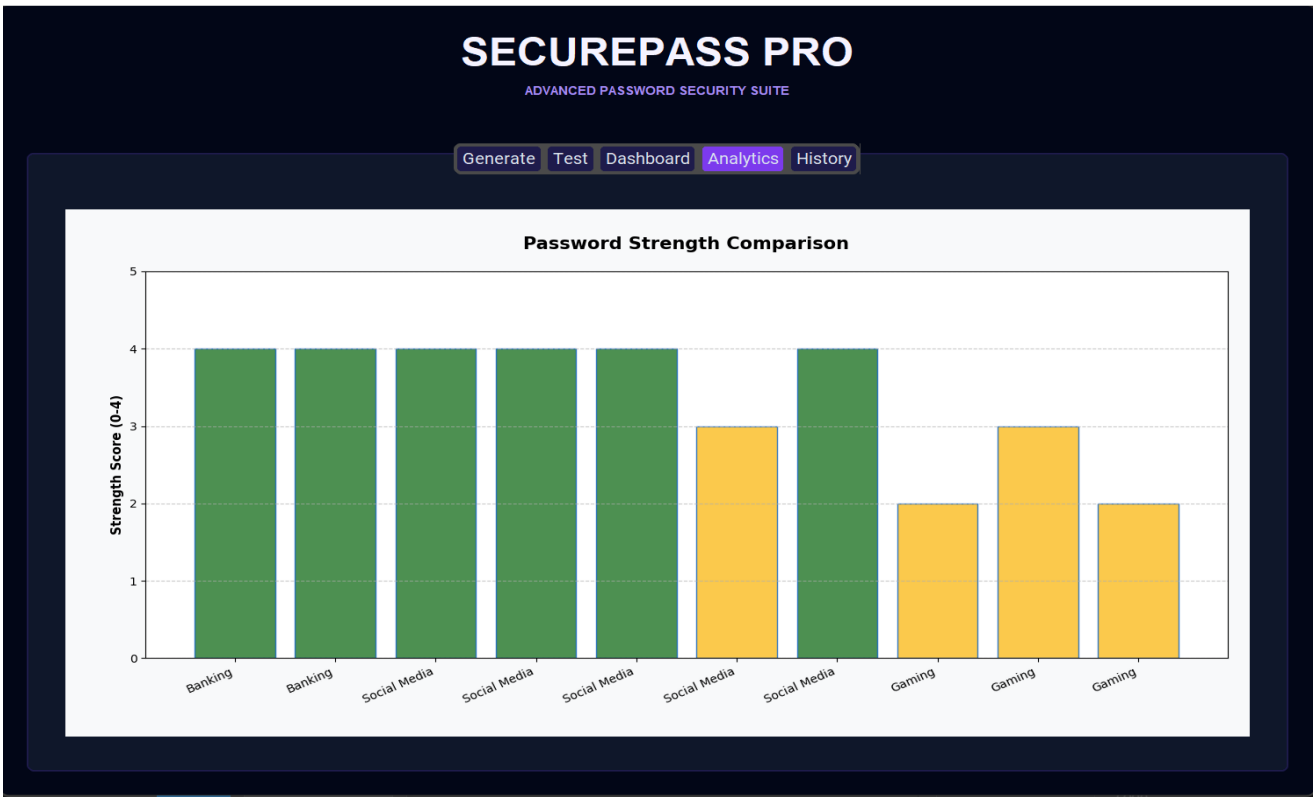




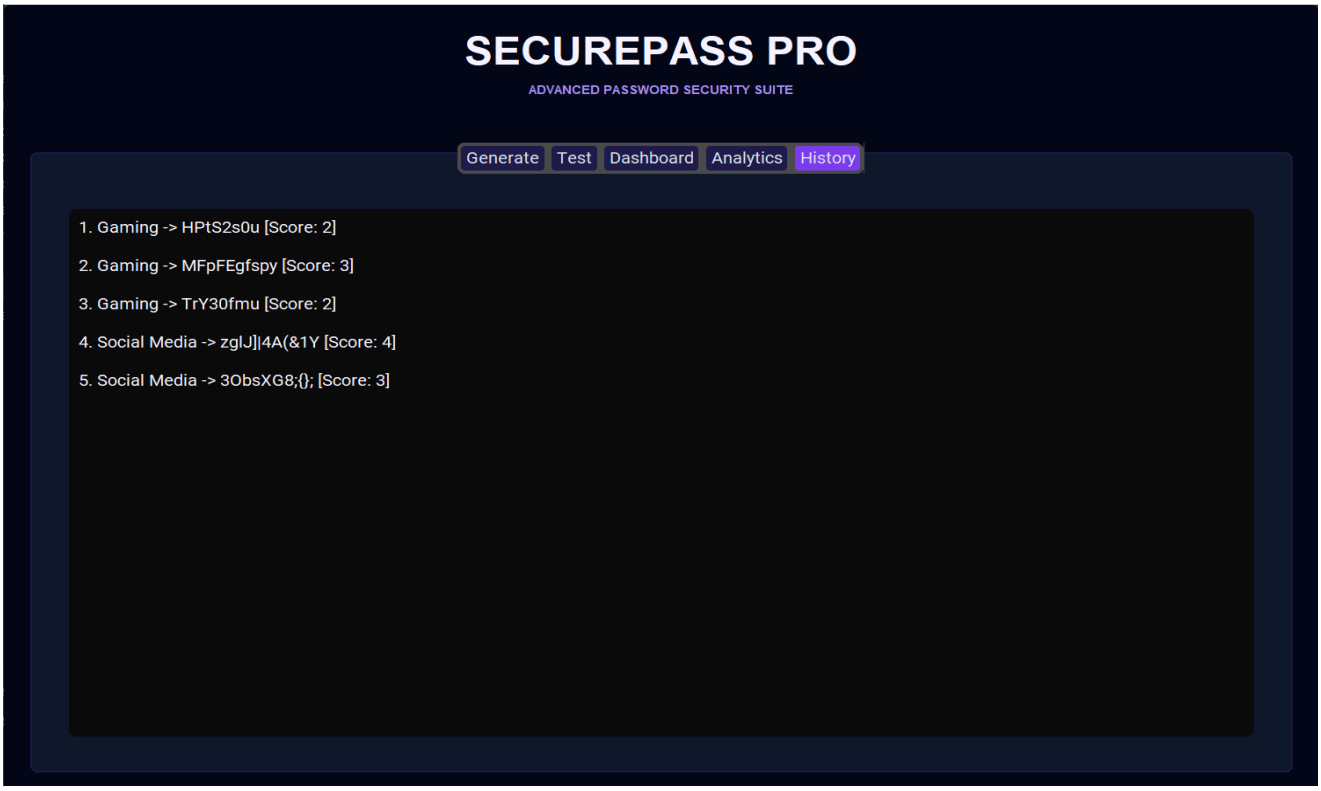
Test & DashBoard



Platform	Length	Score	Crack Time
Banking	16	4	centuries
Banking	20	4	centuries
Social Media	12	4	3 years
Social Media	12	4	3 years
Social Media	12	4	3 years
Social Media	11	3	4 months
Social Media	12	4	3 years
Gaming	8	2	3 hours
Gaming	10	3	12 days
Gaming	8	2	3 hours



Analytics & History



8. CONCLUSION & SCOPE FOR EXTENSION

Conclusion: SecurePass Pro successfully bridges the gap between simple random string generators and professional security tools. By integrating advanced libraries like zxcvbn and matplotlib, the project demonstrates how Python can be used to create modular, data-driven applications. The use of Object-Oriented Programming (OOP) ensures that the code is maintainable and that the UI remains responsive even when performing complex mathematical entropy calculations.

Scope for Mini Project Extension: As per the alignment guidelines, this project serves as a foundational module for a full-scale security suite. Future extensions for the Mini Project phase include:

1. **Persistent Storage:** Transitioning from in-memory session history to an encrypted **SQLite Database** (AES-256) to allow users to save their passwords permanently.
2. **User Authentication:** Implementing a **Master Password** login system using **PBKDF2** password hashing to protect the vault.
3. **Automatic Breach Detection:** Integrating an API (like *Have I Been Pwned*) to check if generated or manual passwords have appeared in known data breaches.

---

9. REFERENCES

1. **zxcvbn-python documentation:** Realistic password strength estimation based on Dropbox's algorithm.
2. **CustomTkinter Documentation:** Modern UI components and high-DPI scaling in Python.
3. **Matplotlib User Guide:** Embedding figures in Tkinter using the FigureCanvasTkAgg backend.
4. **Python 'secrets' Module:** Cryptographically strong random numbers for managing secrets.

5. Wheeler, D. (2012). zxcvbn: Low-Budget Password Strength Estimation. Dropbox Engineering. Available at: https://github.com/dwolfhub/zxcvbn-python
6. Lütkemeyer, T. customtkinter Documentation – Modern UI Components for Tkinter. Available at: https://customtkinter.tomschimansky.com/
7. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.
8. Python Software Foundation. secrets — Generate secure random numbers for managing secrets. Python 3 Documentation. https://docs.python.org/3/library/secrets.html
9. Python Imaging Library (Pillow). https://pillow.readthedocs.io/